



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Татјана Гавриловић

**Пројектовање и имплементација
текстуалног претраживача над
транскриптима Youtube подкаста на
српском језику**

ЗАВРШНИ РАД

Мастер академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Татјана Гавриловић	Број индекса:	R2-38/2023
Степен и врста студија:	Мастер академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Бранко Милосављевић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Пројектовање и имплементација текстуалног претраживача над транскриптима Youtube подкаста на српском језику

ТЕКСТ ЗАДАТКА:

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.
--

Руководилац студијског програма:	Ментор рада:

Примерак за: □ - Студента; □ - Ментора
--



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - мастер рад
Аутор, АУ:	Татјана Гавриловић
Ментор, МН:	Др Бранко Милосављевић, редовни професор
Наслов рада, НР:	Пројектовање и имплементација текстуалног претраживача над транскрипцијама Youtube подкаста на српском језику
Језик публикације, ЈП:	српски/ћирилица
Језик извода, ЈИ:	српски/енглески
Земља публикавања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	11/56/33/1/19/0/2
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	Шаблон, завршни рад, упутство
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	Овај документ представља упутство за писање завршних радова на Факултету техничких наука Универзитета у Новом Саду. У исто време је и шаблон за Typst.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	01.01.2025
Чланови комисије, КО:	Председник: Др Петар Петровић, ванредни професор
	Члан: Др Марко Марковић, доцент
	Члан:
Члан, ментор:	Др Бранко Милосављевић, редовни професор
	Потпис ментора



UNIVERSITY OF NOVI SAD • **FACULTY OF TECHNICAL SCIENCES**
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	
Author, AU :	Tatjana Gavrilović
Mentor, MN :	Igor Dejanović, Phd., full professor
Title, TI :	Design and implementation of a text search engine over transcripts of YouTube podcasts in the Serbian language
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian/English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	11/56/33/1/19/0/2
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	Template, thesis, tutorial
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad
Note, N :	
Abstract, AB :	This document provides guidelines for writing final theses at the Faculty of Technical Sciences, University of Novi Sad. At the same time, it serves as a Typst template.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	01.01.2025
Defended Board, DB :	President: Petar Petrović, Phd., assoc. professor
	Member: Marko Marković, Phd., asist. professor
	Member:
Member, Mentor:	Igor Dejanović, Phd., full professor
	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА**
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
2	Изазови обраде српског језика као природног језика	3
2.1	Историјски разлози за ограничене језичке ресурсе	3
2.2	Писмо, дијалекти и фонолошке варијанте	4
2.3	Морфолошка сложеност српског језика	4
2.4	Вишезначност и семантичка сложеност	5
3	Преглед приступа	7
4	Архитектура система	9
4.1	База података — <i>PostgreSQL</i> и <i>pgvector</i>	9
4.2	Претраживачки систем — <i>Elasticsearch</i>	9
4.3	Позадински сервис — <i>FastAPI</i>	10
4.4	Кориснички интерфејс — <i>Angular</i>	10
4.5	Мрежа и покретање система	11
5	Транскрипција и припрема корпуса	13
5.1	Обим корпуса	13
5.2	Прикупљање и транскрипција	13
5.3	Формат складиштења	14
5.4	Складиштење у <i>PostgreSQL</i> бази	15
5.5	Модел података	15
5.6	Функција за увоз података у базу	16
6	Лексичка претрага (<i>Elasticsearch</i>)	19
6.1	Шта је <i>Elasticsearch</i>	19
6.2	<i>Elasticsearch</i> наспрам релационе базе података	19
6.3	Дистрибуирана архитектура	19
6.4	Како <i>Elasticsearch</i> чува и претражује податке?	20
6.5	BM25 и рангирање резултата	20
6.6	Ниво вршења индексирања	21
6.7	Мапирање	21
6.8	Изазови српског језика у <i>Elasticsearch</i> -у	23
6.9	Анализатор	23
6.10	Алгоритам индексирања	26
6.11	Алгоритам претраге	29
7	Семантичка (векторска) претрага	33
8	Хибридизација резултата претраге (RRF)	35
9	Евалуација система	37
10	Изглед апликације	39
11	Закључак	41
	Списак слика	43
	Списак листинга	45
	Списак табела	47
	Списак коришћених скраћеница	49
	Списак коришћених појмова	51
	Биографија	53
	Литература	55

Енорман раст и популаризација подкаста (енг. *podcasts*) и YouTube садржаја су резултат комбинације технолошке демократизације, промјене навика корисника, те развоја нових начина дистрибуције и зараде. Међутим, док количина доступног садржаја континуирано расте, могућност прецизног проналажења релевантних тренутака унутар дугих епизода заостаје за овим развојем. Корисник који тражи конкретну информацију у епизоди тренутно нема начин да избјегне преслушавање читавог садржаја, нити да добије хронолошки организован преглед тема унутар епизоде са могућношћу директног преласка на релевантан исјечак.

Слична рјешења овог проблема рађена су над корпусом енглеског језика. Платформа *Spotify* је развила семантичку претрагу подкаста засновану на моделу обраде природног језика (ОПЈ, енгл. *Natural Language Processing, NLP*) и приближној претрази (к) најближих сусједа (енгл. *Approximate Nearest Neighbor, ANN*) [1], ипак ријеч је искључиво о семантичкој претрази, не о хибридном приступу који комбинује семантичку и лексичку претрагу. Пројекат *TREC Podcasts Track* је представљао истраживачки оквир за претрагу подкаст сегмената, покренут 2020. године и настављен 2021. године [2], али је обустављен, а одговарајући скуп података је укинут 2023. године [3].

Иако постоје примјери примјене семантичке претраге на српском језику у другим доменима (нпр. правна пракса [4]) и активан истраживачки рад на текстуалним *NLP* ресурсима за српски (нпр. *COMtext.SR* [5]), ни у једном пронађеном извору није идентификовано рјешење за семантичку или хибридну претрагу говорног, односно подкаст садржаја на српском језику са мапирањем на конкретан тренутак у оквиру епизоде.

Посебан изазов представља и сам српски језик, који припада језицима са ограниченим ресурсима у области обраде природног језика. Доступност квалитетних и обимних корпуса за српски знатно је мања у односу на доминантне језике. Због тога вишејезични модели, кроз пренос знања из других језика, не могу у потпуности обухватити језичке специфичности српског језика, укључујући његову морфосинтаксичку сложеност и различите језичке варијанте [6].

Циљ овог рада је дизајн инфраструктуре и имплементација система за хибридну претрагу *YouTube* садржаја на српском језику, који комбинује лексичку и семантичку претрагу путем *Reciprocal Rank Fusion* методе, уз очување грануларности на нивоу једног или више сегмената ради навигације до тачног или приближног тренутка у епизоди.

Сљедеће поглавље разматра специфичне изазове обраде српског језика као природног језика, са освртом на историјске околности развоја језичких ресурса, те његово морфолошко богатство које утиче на изградњу система за претрагу. Треће поглавље даје преглед цјелокупног приступа и представља основне токове система — од припреме и индексирања података до обраде корисничког упита и добијања коначних резултата — чиме се пружа оквир за детаљнији опис у наредним поглављима. У глави четири описана је архитектура система, коришћене технологије и организација контејнеризованог окружења. Глава пет посвећена је транскрипцији и припреми корпуса, укључујући поступак добијања транскрипата, тако и дизајн структуре података чуване у релационој бази. У шестом поглављу описује се лексичка претрага заснована на *Elasticsearch*-у, док се поглавље седам бави семантичком, односно векторском претрагом. Глава осам представља поступак хибридизације резултата лексичке и семантичке претраге коришћењем алгорита *Reciprocal Rank Fusion (RRF)*, тиме би главу девет посветили евалуацији система. Поглавље десет приказује кориснички интерфејс и начин интеракције корисника са системом. Закључним, једанаестим поглављем, заокружиће се предмет овог рада, сумираће се постигнути резултати и указати на могуће правце даљег развоја.

Глава 2

Изазови обраде српског језика као природног језика

Обрада природних језика (енг. *Natural Language Processing, NLP*) се бави проучавањем метода за рачунарску обраду и интерпретацију текстуалних података написаних на неком од природних језика, тј. језика које људи користе у међусобној комуникацији (нпр. српски језик, енглески, итд.). За разлику од програмских језика, код којих су начини изражавања унапријед строго дефинисани, природни језици немају таквих инхерентних ограничења [7].

Развој *NLP* технологија дуго је био усмјерен првенствено на енглески језик, који и данас има доминантну улогу у *NLP* истраживањима [8]. Ова ситуација је дјелимично посљедица доминантне улоге енглеског језика у међународној комуникацији, што је довело и до веће комерцијалне заинтересованости за развој *NLP* рјешења за енглеско говорно подручје [7]. Посљедице су посебно изражене код језика попут српског, за које не постоје ресурси по обиму и квалитету упоредиви са онима доступним за енглески језик. Недостатак великих и језички аотираних корпуса отежава развој и евалуацију *NLP* модела, како класичних статистичких метода, тако и савремених дубоких неуронских мрежа [6].

Корпус (мн. корпуси) представља велику, структурисану збирку аутентичних језичких података — писаних или говорних — представљених у машински читљивом формату, намјенски одабраних и узоркованих тако да буду репрезентативни за одређени језик, и често аотираних језичким метаподацима [6]. Управо величина и квалитет доступног корпуса директно одређују колико поуздано *NLP* систем може да ради над датим језиком — питање које је, како ће бити показано у наставку овог поглавља, за српски језик посебно изражено.

2.1 Историјски разлози за ограничене језичке ресурсе

Мали обим доступних дигиталних ресурса за српски језик није случајност. Оно је посљедица вишевијековног низа историјских околности које су директно утицале на очување писаног наслеђа.

Српски је индоевропски језик који је развио богату средњовјековну књижевну традицију, укључујући темељна дјела попут Мирослављевог јеванђеља (око 1180.), Вукановог јеванђеља (око 1202.), Светосавског номоканона (1219.) и Душановог законика (1349.). Међутим, османска окупација (средина 15. – почетак 19. вијека) разорила је ову традицију. Манастирске библиотеке, централне за производњу рукописа, суочиле су се са уништавањем и пљачком, при чему су рукописи спаљивани, крадени или продавани колекционарима у Русији, Аустрији или Венецији¹ [6].

Након ослобођења од османске окупације (1804–1815), Србија је постепено обнављала своју књижевну и културну инфраструктуру, али је њено текстуално наслеђе наставило да трпи ударце. Балкански ратови (1912–1913) и Први свјетски рат изазвали су значајна разарања, док је најкатастрофалнији губитак уследио 1941. године, када су њемачке бомбе уништиле Народну библиотеку Србије, уништивши преко 500.000 томова, укључујући 1.424 средњовјековна ћирилична рукописа [6]. Ови губици су дугорочно осиромашили српске језичке и књижевне ресурсе, што данас отежава развој језичких технологија. Недостатак података додатно се продубљује ограничењем дигитализацијом, рестриктивним ауторским правима и недовољном институционалном подршком [6].

¹На примјер, Вуканово јеванђеље данас се налази у Руској националној библиотеци, док је Минхенски српски псалтир однесен у Баварску 1688. године.

2.2 Писмо, дијалекти и фонолошке варијанте

Још један историјски развој са савременим значајем јесте језичка модернизација српског језика током постосманског културног препорода. Српски филолог и етнолог Вук Караџић предводио је свеобухватну језичку реформу почетком 19. вијека, трансформишући не само српски, већ и хрватску језичку стандардизацију. Ово је утицало на будућност српскохрватског језика и ширег јужнословенског језичког простора, при чему је Бечки књижевни договор из 1850. године — који су потписали српски, хрватски и словеначки интелектуалци — прогласио заједнички језик са два писма, ћирилицом и латиницом [6].

Насљеђе овог историјског развоја директно се одражава на језичке особине са којима се овај рад суочава: српски језик карактерише паралелна употреба два писма — ћирилице и латинице — која се у савременој пракси, укључујући и садржај анализиран у овом раду, слободно смјењују, често и унутар истог документа или говорника [6]. Поред писма, српски посједује и двије фонолошке варијанте изговора старог словенског гласа јат (ѣ) — екавску (нпр. „дете“, „лепо“) и ијекавску (нпр. „дијете“, „лијепо“). Додатно, српски језик обухвата пет дијалеката (призренско-тимочки, косовско-ресавски, шумадијско-војвођански, зетско-јужносанџачки и источнохерцеговачки), те носи препознатљиве морфосинтаксичке особине, попут богате падежне флексије и слободног реда ријечи у реченици [6].

Разлике између дијалеката могу довести до различитих облика истих ријечи у тексту. То је посебно релевантно за призренско-тимочки дијалекат јужне Србије, у којем се, услед историјских језичких контаката на простору Балканског језичког савеза, испољавају морфосинтаксичке варијације повезане са утицајем сусједних језика, укључујући бугарски и македонски. У односу на стандардни српски, нестандартне варијанте призренско-тимочног дијалекта могу показивати различите површинске облике, што додатно отежава њихову аутоматску обраду и претрагу [9]. Уколико би се међу говорницима у корпусу нашли и говорници оваквих регионалних варијетета, прилагођени анализатор развијен за стандардни српски (поглавље 6) не би нужно исправно нормализовао такве облике. Ова могућност није посебно испитивана у оквиру овог рада — наводи се као теоретски, а не емпиријски потврђен изазов. На примјер, у призренско-тимочким говорима јавља се облик „работим“, који се разликује од стандардног српског облика „радим“, иако има исто значење. Овакве разлике могу отежати претрагу текста уколико систем није прилагођен дијалекатским варијантама.

Варијабилност писма имала је директан утицај на имплементацију система описаног у овом раду. И претрага у *Elasticsearch*-у (поглавље 6) и обрада текста прије векторског уграђивања (поглавље 7) захтијевале су нормализацију писма, односно превођење садржаја у јединствени облик (латиницу) прије индексирања. На тај начин се исти појам, без обзира на то да ли је написан ћирилицом или латиницом, третира као један термин, а не као два различита облика.

Латинично писмо српског језика користи дијакритичке знакове, односно додатне ознаке којима се основно слово разликује од другог слова. У српском језику такве знакове имају слова ч, ћ, ш, ж и ђ у латиничном облику: *č, ć, š, ž* и *đ*. Ћирилица за исте гласове има посебна слова, па се они у њој не записују помоћу дијакритичких знакова. Ова разлика је значајна за претрагу јер исти појам може бити записан са дијакритичким знаковима или без њих.

Због тога је нормализација дијакритика укључена у оба дијела система. У оквиру лексичке претраге (поглавље 6), њихова обрада реализована је кроз прилагођени анализатор заснован на ICU додатку [[10]]. У припреми текста за векторско уграђивање (поглавље 7), нормализација дијакритика представљала је један од првих корака обраде транскрипта, одвојен од конверзије ћирилице у латиницу. На тај начин се смањује број различитих облика истог појма и омогућава његово препознавање без обзира на начин записивања.

2.3 Морфолошка сложеност српског језика

Морфологија представља дио граматике који се бави врстама ријечи, њиховом структуром и промјеном њихових облика [11]. У српском језику посебно су значајне промјенљиве врсте ријечи, код којих се облик

мијења у зависности од различитих граматичких категорија. Међу њима се издвајају именице, придјиви и глаголи, чија морфолошка својства доприносе великој варијабилности површинског облика текста.

Именице имају граматичке категорије рода, броја и падежа. Њихова промјена по падежима назива се деклинација, при чему једна лексема може имати више различитих облика. Лексема је основна самостална јединица лексичког система која обухвата све граматичке облике и различита значења која једна ријеч може остварити [12]. Српски језик има седам падежа: номинатив, генитив, датив, акузатив, вокатив, инструментал и локатив. На примјер, именица град може се појавити у облицима град, града, граду, град, граде, градом, граду. Иако се површински облици разликују, они представљају различите облике исте лексеме. Именице припадају једном од три граматичка рода — мушком, женском или средњем.

Придјиви се са именицом уз коју стоје слажу у роду, броју и падежу. Због тога и они мијењају свој облик у зависности од граматичког контекста. На примјер, придјив млад јавља се у облицима млад човјек, млада жена, младо дијете, а кроз падеже у конструкцији млад човјек, младог човјека, младом човјеку, младим човјеком. Осим тога, код описних придјева постоји и могућност степеновања, нпр. висок, виши, највиши, што представља додатни извор морфолошке варијабилности.

Глаголи имају сложенији систем морфолошких категорија. Њихови облици зависе од лица и броја, времена, начина и глаголског вида, а српски језик има и различите неличне глаголске облике. Тако се глагол радити може појавити као радим, радиш, ради, радимо, радите, раде, док се кроз различита времена и облике јављају, на примјер, радио је, радиће, радио бих, радећи. Посебну категорију представљају глаголски придјиви, укључујући радни (радио, радила, радило) и трпни (урађен, урађена, урађено). Глаголе такође карактерише глаголски вид, при чему се разликују свршени и несвршени глаголи, нпр. писати – написати.

Оваква морфолошка разноврсност значи да се иста лексема у тексту може појавити у великом броју различитих површинских облика. То представља важан изазов за аутоматску обраду и претрагу текста на српском језику, јер облик који се појављује у корисничком упиту не мора бити идентичан облику исте лексеме који се налази у тексту.

Дио ове варијабилности ријешен је у оквиру лексичке претраге описане у поглављу 6, гдје прилагођени анализатор, уз стематизацију (*serbian_stemming*), своди различите облике исте лексеме на заједнички стем. Стематизација представља поступак свођења различитих облика ријечи на њихову заједничку основу, односно стем, који се добија уклањањем одређених наставака и дијелова ријечи и не мора представљати стварну ријеч у језику. На тај начин се, на примјер, различити падежни облици исте именице могу повезати у претрази. Ипак, стематизација не може повезати ријечи које имају сродно значење, али различиту основу, као што су писати и написати. Оне представљају различите лексеме, па их стематизација не може повезати само на основу њихове семантичке сличности. Управо тај преостали дио варијабилности представља један од разлога за увођење семантичке, односно векторске претраге, описане у поглављу 7.

2.4 Вишезначност и семантичка сложеност

Поред морфолошке флексије, додатан проблем представља чињеница да се текстови на природним језицима често одликују великим степеном комплексности, као и нејасноћама и двосмисленостима, односно вишезначностима у изражавању [7]. Исти концепт може бити изражен различитим ријечима или формулацијама, што представља посебан изазов за лексичку претрагу. На примјер, корисник може претраживати појам „додаци исхрани“, док се у транскрипту подкаста говори о „суплементима“. Иако се ове формулације разликују на нивоу ријечи, њихово значење је блиско. Лексичка претрага заснована на подударану термина такву везу не може поуздано препознати, осим ако су одговарајуће ријечи унапријед наведене као синоними. Ово ограничење представља један од главних разлога за увођење семантичке, односно векторске претраге (поглавље 7), која омогућава поређење текстова на основу њиховог значења, а не само на основу подударања конкретних ријечи.

Систем описан у овом раду реализован је као скуп независних сервиса, међусобно повезаних преко заједничке мреже и покренутих помоћу Docker Compose-а, који представља алат за дефинисање и покретање апликација састављених од више контејнера једном командом [13]. Контејнеризација омогућава да сваки сервис (база података, претраживачки систем, позадински сервис, кориснички интерфејс) буде изолован у сопственом окружењу, са тачно дефинисаним зависностима, при чему се цијели систем покреће и зауставља на исти начин, без обзира на то да ли се извршава на развојној машини или серверу. Систем чине пет сервиса на заједничкој мрежи (енг. *app-network*): база података (*PostgreSQL* са *pgvector* екстензијом), административни алат за базу (*pgAdmin4*), претраживачки систем (*Elasticsearch*), позадински сервис (*FastAPI*) и кориснички интерфејс (*Angular*).

4.1 База података — *PostgreSQL* и *pgvector*

PostgreSQL у оквиру система представља централну базу података и извор истине (енг. *source of truth*), главни и поуздани извор за све структуриране податке. У њему се чувају метаподаци о каналима, епизодама и епизодним сегментима. Поред тога, захваљујући екстензији *pgvector*, у бази се чувају и векторске репрезентације текста (енг. *embeddings*), које се користе за семантичку претрагу описану у поглављу 7. *pgvector* је екстензија отвореног кода која *PostgreSQL*-у додаје подршку за чување вектора и претрагу по њиховој сличности [14]. Разлози за избор *pgvector*-а у односу на друга рјешења, као што су *FAISS*, *Qdrant*, *Milvus* и *Pinecone*, детаљно су образложени у поглављу 7.

За покретање базе користи се службена слика (енг. *image*) ***pgvector/pgvector:pg16***, која садржи *PostgreSQL* 16 са уграђеном *pgvector* екстензијом. Подаци се чувају у именованом простору за трајно чување података (енг. *volume*), чиме се обезбјеђује њихово очување и након поновног покретања контејнера. Спремност сервиса провјерава се помоћу *healthcheck*-а, који користи команду *pg_isready*. Конфигурација *PostgreSQL* базе са *pgvector* екстензијом се може погледати у листингу 1.

```
postgres:
  image: pgvector/pgvector:pg16
  environment:
    - POSTGRES_DB=podcast_search
    - POSTGRES_USER=podcast_user
    - POSTGRES_PASSWORD=podcast_password
  volumes:
    - postgres_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U podcast_user"]
```

Листинг 1: Конфигурација *PostgreSQL* базе са *pgvector* екстензијом

За административни приступ бази података, који укључује преглед табела и ручно извршавање упита, коришћен је *pgAdmin4*. Административном интерфејсу приступа се путем веб интерфејса на порту 5050.

4.2 Претраживачки систем — *Elasticsearch*

Elasticsearch сервис је заснован на прилагођеној *Docker* слици, изграђеној на службеној слици ***elasticsearch:8.13.2***. Слици је додат *analysis-icu* додатак [10], који је коришћен за нормализацију и обраду ћириличног и латиничног текста, што је детаљније описано у поглављу 6. Прилагођена слика се дефинише следећим *Dockerfile*-ом, приказаним на листингу 2.

```
FROM docker.elastic.co/elasticsearch/elasticsearch:8.13.2
RUN bin/elasticsearch-plugin install --batch analysis-icu
```

Листинг 2: *Dockerfile* за изградњу прилагођене *Elasticsearch* слике са *analysis-icu* додатком

Сервис ради у режиму *single-node*, што значи да се *Elasticsearch* извршава као један чвор, без повезивања са другим чворовима у кластеру. *X-Pack* безбједносни механизми [15], који обезбјеђују аутентификацију и контролу приступа *Elasticsearch*-у, су искључени, што одговара развојном и интерном окружењу у којем је систем коришћен. Спремност сервиса провјерава се помоћу *healthcheck*-а, који чека да статус кластера више не буде *red*. Комплетна конфигурација индекса, анализатора и поступака индексирања и претраге описана је у поглављу 6.

4.3 Позадински сервис — *FastAPI*

Позадински дио система реализован је кроз *Python 3.11*, уз коришћење оквира *FastAPI* и *Uvicorn* сервера, који имплементира *ASGI* (енг. *Asynchronous Server Gateway Interface*) стандард и служи за покретање апликације и обраду *HTTP* захтјева. *FastAPI* је изабран због једноставне имплементације *API*-ја. Поред тога, *Python* пружа развијен екосистем за машинско учење и обраду природног језика [16], који је у овом систему коришћен и за генерисање векторских репрезентација помоћу библиотеке *sentence-transformers*.

За рад са базом података користи се *SQLAlchemy* као *ORM* (енг. *Object-Relational Mapping*), уз *psycopg2-binary* као *PostgreSQL* управљачки програм. Промјене структуре базе управљају се помоћу *Alembic*-а, док се за рад са векторским колонама користи *Python* подршка за *pgvector*. Валидација конфигурације и података реализована је помоћу библиотека *Pydantic* и *pydantic-settings*, док се вриједности из окружења учитавају помоћу библиотеке *python-dotenv*. Комуникација са *Elasticsearch*-ом остварује се преко службеног *Python* клијента.

Docker слика позадинског сервиса гради се у једној фази, уз коришћење кеширања слојева. Зависности наведене у датотеци *requirements.txt* инсталирају се прије копирања изворног кода, што омогућава да се приликом измјене кода поново искористи постојећи слој са инсталираним зависностима, без њихове поновне инсталације. Модели преузети преко библиотека *Hugging Face*, укључујући модел за генерисање векторских репрезентација, чувају се у именованом *volume*-у (*hf_cache*). На тај начин се избјегава њихово поновно преузимање приликом поновног покретања контејнера.

4.4 Кориснички интерфејс — *Angular*

Кориснички интерфејс реализован је у *Angular*-у [17], а за његову изградњу користи се *Docker multi-stage build* [18], који раздваја фазу изградње апликације од фазе њеног покретања. У првој фази, заснованој на слици *node:22-alpine*, гради се продукцијска верзија апликације, док се у другој фази добијени статички фајлови копирају у минималну *nginx:alpine* слику. На тај начин коначна слика садржи само фајлове неопходне за покретање апликације, без изворног кода и алата који су потребни само током изградње, чиме се смањује њена величина.

Angular апликација реализована је као *SPA* (енг. *Single Page Application*), што значи да се рутирање између страница обавља на страни клијента. Ово је посебно важно за директан приступ конкретној епизоди и одговарајућем тренутку у транскрипту. Због тога је *nginx* подешен тако да захтјеве за путање које не представљају постојеће статичке фајлове прослиједи апликацији преко *index.html*, приказано у листингу 3.

```
location / {
    try_files $uri $uri/ /index.html;
}
```

Листинг 3: Конфигурација *nginx*-а за просљеђивање захтјева *Angular SPA* апликацији

Овакво подешавање омогућава да директан приступ одређеној рути или освјежавање странице функционисе исправно, јер *nginx* захтјев просљеђује *Angular*-овом механизму за рутирање умјесто да врати грешку 404. Додатно, *nginx* је подешен за компресију одговора помоћу *gzip*-а и кеширање статичких ресурса, као

што су *JavaScript* и *CSS* фајлови, чиме се смањује количина података која се преноси приликом поновног учитавања апликације.

4.5 Мрежа и покретање система

Сви сервиси комуницирају преко заједничке *Docker bridge* мреже (*app-network*), што им омогућава међусобну комуникацију преко имена сервиса (нпр. позадински сервис приступа бази преко хоста *postgres*, а не *localhost-a*), без потребе за излагањем интерних портова ван контејнера. Покретање и заустављање, као и потпуно брисање базе приказане су у листингу 4 кроз њихове команде.

```
# Покретање свих сервиса у позадини уз поновну изградњу слика
docker-compose up -d --build

# Заустављање и уклањање контејнера уз задржавање података у volume-има
docker-compose down

# Заустављање и уклањање контејнера и именованих volume-a
docker-compose down -v
```

Листинг 4: Команде за покретање и заустављање *Docker Compose* сервиса

Глава 5

Транскрипција и припрема корпуса

Ово поглавље описује поступак прикупљања и припреме корпуса који представља основу за претрагу описану у наредним поглављима. Најприје је представљен обим и извор прикупљених података, а затим је описан поступак транскрипције видео садржаја. Након тога, објашњен је начин организације и складиштења добијених података на диску, као и њихов увоз у релациону базу података која се користи у систему.

5.1 Обим корпуса

Корпус коришћен у овом раду прикупљен је са 32 *YouTube* канала на српском језику, који обухватају различите тематске области и формате, укључујући интервјуе, друштвено-политичке подкасте, едукативни и научно-популарни садржај, као и забавне формате. Укупно је прикупљено и транскрибовано 15.889 епизода.

Избор канала обухвата разноврсне теме, говорнике и стилове комуникације, чиме се обезбјеђује хетерогеност корпуса и омогућава испитивање система у условима различитим од оних који би били присутни у једној уско дефинисаној тематској области. На тај начин корпус пружа погодну основу за процјену способности система да претражује садржај различитих врста и тематских усмјерења. Табела 1 приказује преглед свих канала обухваћених корпусом, заједно са бројем епизода прикупљених са сваког канала.

Табела 1: Списак канала у корпусу и број транскрибованих епизода по каналу.

Канал	Број епизода	Канал	Број епизода
Bez Cenzure	3.345	RTS Образовно-научни програм	2.105
RTS Око	1.725	Angelast	1.073
Pojačalo	701	Priče U Nama	701
RTS TV serije	828	RTS Ordinacija	789
Da sam ja neko	656	Nikola Radin Podcast	501
Ivan Kosogor Podcast	353	RTS Kvadratura kruga	353
Catena mundi	340	Biznis Priče	324
Novi Standard	289	Jao Mile	244
Još podkast jedan	234	Centar za antiautoritarne studije	157
Rubikon	119	Tampon zona	119
Vredno priče	115	Miljanov korner	102
Sto minuta buke	168	Dva i po psihijatra	90
Lukavstvo uma	85	Izokrenuta Učionica	76
Reketiranje	75	Sada i овде	72
Drugačiji fitness	49	Kontranapad	46
Zalet inovacija	29	Ćao Nevena	26

5.2 Прикупљање и транскрипција

Видео садржај преузет са *YouTube* канала било је потребно претворити у текстуални облик погодан за даљу обраду. У ту сврху коришћена је технологија аутоматског препознавања говора (енгл. **Automatic Speech Recognition**, ASR), која представља област вјештачке интелигенције усмјерену на претварање говорног

сигнала у текстуални облик. Један од савремених система заснованих на овој технологији је **Whisper**, модел за аутоматско препознавање и транскрипцију говора који је развила компанија OpenAI. Модел је обучен на великом вишејезичном скупу аудио података и намијењен је аутоматској транскрипцији говорног садржаја, уз подршку за различите језике и облике говора [19]. Наведене карактеристике биле су посебно значајне за избор **Whisper**-а у овом раду, будући да се обрађује говорни садржај различитих говорника, при чему се могу јавити варијације у изговору, акценту, интонацији и присуство позадинске буке.

5.3 Формат складиштења

Резултат транскрипције за сваку епизоду складиштен је у *JSONL* (енг. *JSON Lines*) формату — текстуалном формату у којем сваки ред датотеке представља један самосталан и ваљан *JSON* објекат, за разлику од класичног *JSON* формата, у којем цијела датотека мора представљати један објекат или низ објеката.

Овакав приступ омогућава инкрементално читање и обраду датотеке ред по ред, без потребе да се цјелокупна датотека учита у меморију одједном. То је посебно значајно у овом случају, будући да један ред, односно једна епизода, укључује цјелокупан транскрипт са временски означеним (*timestamp*-ованим) сегментима, па датотека која садржи хиљаде епизода једног канала може достићи знатну величину.

Грануларност складиштења је на нивоу канала: **један канал = једна .jsonl датотека**, при чему сваки ред одговара једној епизоди тог канала. Ове датотеке се додатно компресују помоћу *gzip* алгорита (*.gz* екстензија), чиме се значајно смањују потребан простор на диску и вријеме преноса, уз занемарљив трошак декомпресије приликом увоза података у базу.

У листингу 5 је дат скраћени приказ структуре једног реда, односно једне транскрибоване епизоде. Поља *description*, *plain_text* и низ *timestamped_segments* са низом *words* скраћени су ради прегледности. Пуна структура садржи цјелокупан транскрипт, као и временске ознаке за сваку изговорену ријеч.

```
{
  "video_id": "upxT3PGinY0",
  "title": "Branislav Ristivojević – Kosovo i Metohija je ogledalo srpske politike / Rubikon podcast 105",
  "description": "Гост емисије Рубикон био је... [скраћено]",
  "published_at": "2025-06-12T17:01:10Z",
  "duration": "PT1H4M28S",
  "view_count": "58228",
  "like_count": "443",
  "channel_title": "Rubikon",
  "channel_id": "UCb0hTyq2vHyxaimv5cgjAdw",
  "tags": [],
  "file_path": "/var/youtube/rubikon/upxT3PGinY0.webm",
  "download_time": "2025-08-28T20:15:34.711178",
  "url": "https://www.youtube.com/watch?v=upxT3PGinY0",
  "transcription": {
    "plain_text": "Nama nešto opasno je u redu, nama trebaju neki novi filteri... [скраћено]",
    "timestamped_segments": [
      {
        "id": 0,
        "start": 0.0,
        "end": 2.58,
        "text": "Nama nešto opasno je u redu, nama trebaju neki novi filteri.",
        "words": [
          {
            "word": " Nama",
            "start": 0.0,
            "end": 0.16,
            "probability": 0.93
          }
        ]
      }
    ]
  }
}
```

Листинг 5: Примјер структуре JSON записа са метаподацима и транскрипцијом епизоде

Као што се види, један ред садржи метаподатке епизоде (наслов, опис, интернет адресу, датум објаве, број прегледа и лајкова), као и цјелокупан резултат транскрипције — његов садржај (*plain_text*) и низ временски означених сегмената (*timestamped_segments*). *Whisper* у свом изворном излазу пружа и детаљнији ниво грануларности — за сваки сегмент доступан је и низ појединачних ријечи, при чему свака ријеч има сопствени временски опсег и вјероватноћу препознавања (*probability*). Међутим, овај ниво детаља се намјерно не преноси у базу података приликом увоза. Задржавају се само временски опсег (*start, end*) и текст на нивоу цјелокупног сегмента.

Оваква грануларност на нивоу сегмената транскрипта омогућава касније прецизно позиционирање корисника на тачан тренутак у видеу приликом претраге. На тај начин резултат претраге не упућује само на одговарајућу епизоду, већ може корисника усмјерити и на конкретан дио видеа у којем се тражени садржај појављује.

5.4 Складиштење у PostgreSQL бази

Подаци из *.jsonl.gz* датотека увозе се и трајно складиште у PostgreSQL релационој бази података — истој бази. PostgreSQL овдје представља извор истине (*source of truth*) за структуриране податке о каналима, епизодама и сегментима транскрипта. Детаљи о самом сервису, контејнеризацији и екстензији *pgvector* — која се касније користи за векторску претрагу (поглавље 7) — већ су обрађени у поглављу 4.

5.5 Модел података

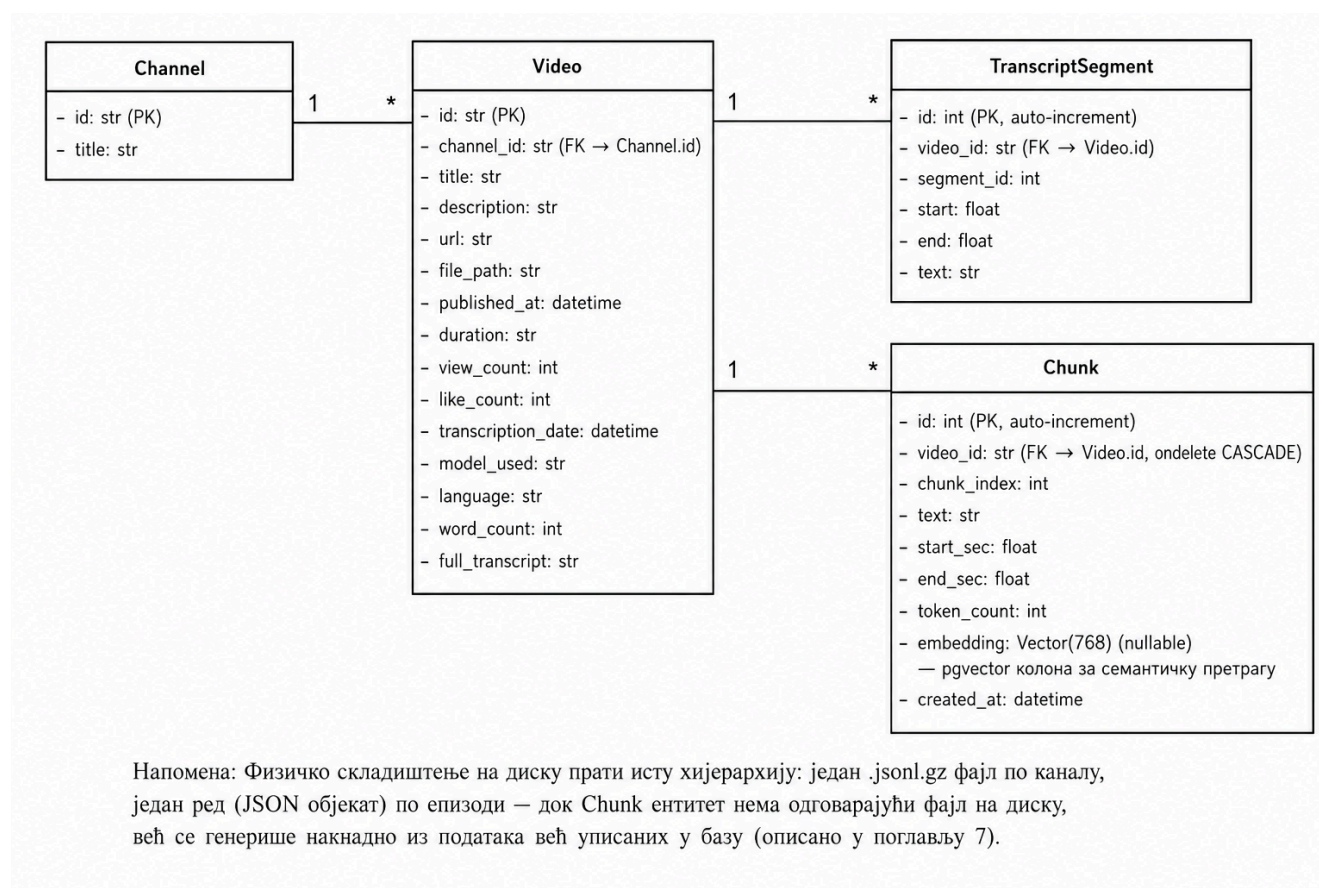
Подаци увезени у базу организовани су кроз четири повезана ентитета, који одражавају природну хијерархију изворних *.jsonl.gz* датотека, уз један додатни ентитет који не потиче директно из *.jsonl* формата, већ се генерише накнадно за потребе семантичке претраге:

- Канал (*Channel*) — један запис за сваки YouTube канал;
- Видео епизода (*Video*) — један запис за сваку транскрибовану епизоду, повезан са тачно једним каналом;
- Сегмент транскрипта (*TranscriptSegment*) — један запис за сваки временски означени дио транскрипта, повезан са тачно једном епизодом;
- *Chunk* — један запис за сваки спојени прозор од једног или два сегмента, намјењен генерисању векторских уграђивања (*embedding*) која се користе у семантичкој, односно векторској, претрази (детаљно објашњено у поглављу 7).

Другим ријечима, структура података на диску — једна *.jsonl.gz* датотека по каналу, један ред по епизоди и више сегмената унутар транскрипта — директно се пресликава у релациони модел. Један канал има више видеа (1:N), док један видео има више сегмената транскрипта (1:N) и, независно од тога, више *chunk*-ова (1:N).

Табела *Chunk* садржи и колону *embedding* типа *Vector(768)* (*pgvector* тип, поглавље 4), као и јединствени (*unique*) индекс над паром *video_id, chunk_index*, који гарантује јединственост редосљеда *chunk*-ова унутар једне епизоде.

Дијаграм класа на слици 1 приказује сва четири ентитета, њихове кључне атрибуте и везе између њих.



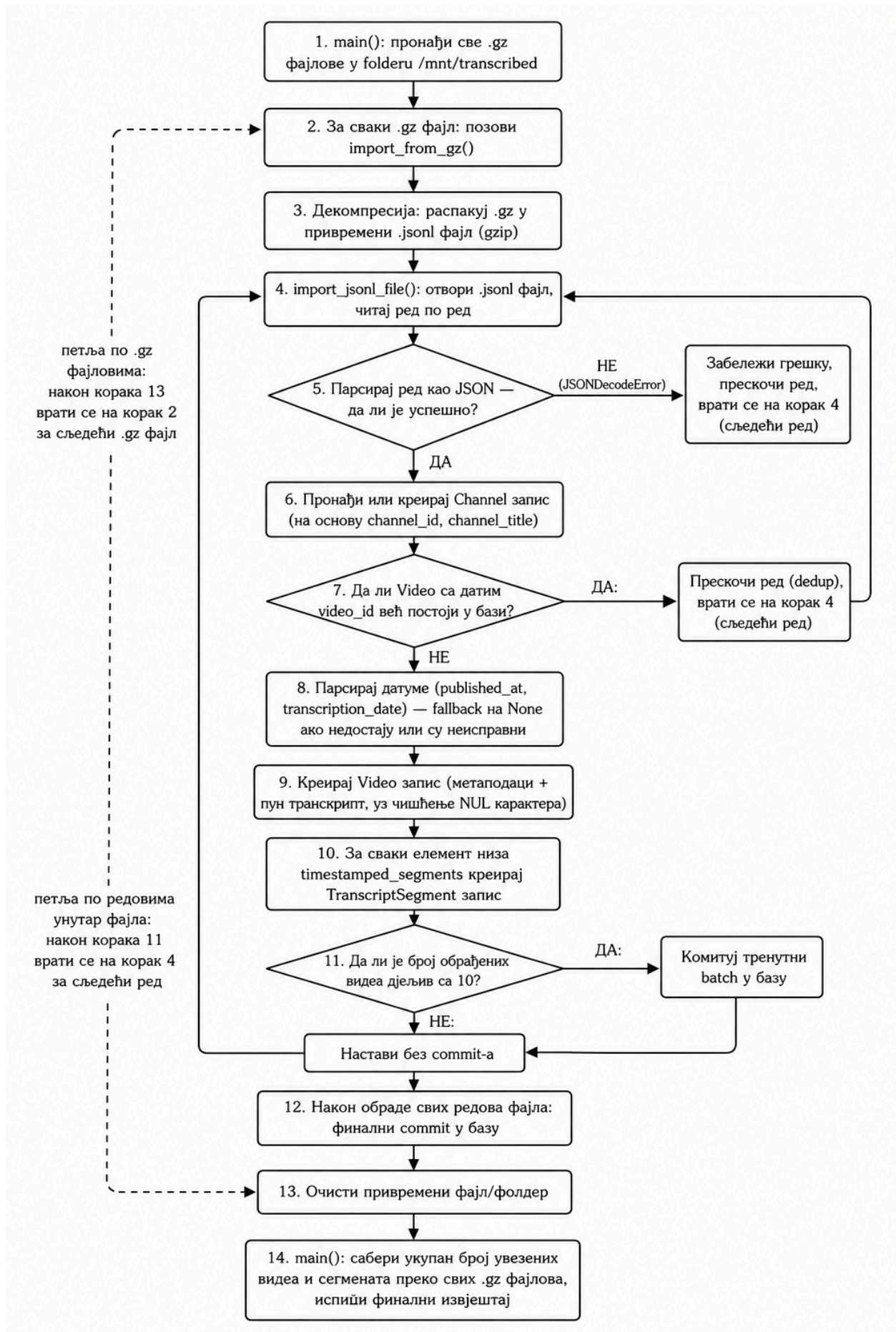
Слика 1: Дијаграм класа модела података: Channel, Video, TranscriptSegment и Chunk.

5.6 Функција за увоз података у базу

Увоз података из *.jsonl.gz* датотека у **PostgreSQL** базу обавља се скриптом *import_data.py*, која се извршава једнократно или по потреби, приликом додавања нових или ажурираних датотека. Скрипта проналази све *.gz* датотеке у фолдеру */mnt/transcribed*, декомпресује их у привремене *.jsonl* датотеке и редом обрађује њихов садржај. Током обраде провјерава се исправност *JSON* записа и постојање видеа у бази, чиме се неисправни редови могу прескочити, а поновни увоз истих података не доводи до дуплирања. За нове записе креирају се одговарајући ентитети **Channel**, **Video** и **TranscriptSegment**, док се промјене периодично потврђују у трансакцији ради ефикаснијег рада. Након обраде свих датотека привремени подаци се уклањају, а скрипта формира завршни извјештај о броју увезених видеа и транскрипционих сегмената. Комплетан ток поступка приказан је на слици 2, а скрипта се покреће унутар *backend* контејнера командом приказаном у листингу 6.

```
docker-compose exec backend python -m app.import_data
```

Листинг 6: Покретање скрипте за увоз података унутар *backend* контејнера

Слика 2: Алгоритам увоза података из *.jsonl.gz* датотека у *PostgreSQL* базу.

Лексичка претрага (*Elasticsearch*)

Лексичка претрага (енг. *lexical search*), позната и као претрага по кључним ријечима (енг. *keyword search*) или пуна-текстуална претрага (енг. *full-text search*), је приступ претраживања текста заснованог на директном подударану термина из корисничког упита са термима садржаним у документима. За разлику од семантичке претраге, која упоређује векторске репрезентације значења (Глава 7), лексичка претрага се ослања на статистичке методе у обради природног језика као што је *TF-IDF* (енг. *Term Frequency-Inverse Document Frequency*) и његовог унапређења *Okapi BM25* (енг. *Best Matching 25, BM25*). *TF-IDF* мјери важност и релевантност одређене ријечи унутар неког документа у односу на цијелу колекцију докумената [20], док *BM25* рангира документе на основу три фактора: учесталости термина упита у документу, ријеткости тог термина у цијелом корпусу (обрнута учесталост документа) и дужине документа у односу на просјечну дужину докумената у колекцији [21]. У оквиру овог рада, лексичка претрага имплементирана је помоћу система *Elasticsearch*.

6.1 Шта је *Elasticsearch*

Elasticsearch је систем намјењен за брзо претраживање и анализу великих количина података у реалном времену. Омогућава претраживање различитих врста података, од обичног текста до структурираних података, а може се користити и за њихову анализу. Његова предност је у томе што омогућава да се велике количине информација претражују брзо и ефикасно, због чега се користи у различитим познатим системима. Википедија (енг. *Wikipedia*) га користи за претрагу текстуалног садржаја и давање приједлога корисницима. Новинарској апликацији, *The Guardian*, служи за праћење реакција публике на нове чланке, док страница *Stack Overflow* помоћу њега проналази слична питања и одговоре. Платформа *GitHub* га користи за претраживање огромне количине програмског кода [22].

6.2 *Elasticsearch* наспрам релационе базе података

Релационе базе података (нпр. *PostgreSQL*) првенствено су намијењене поузданом чувању и обради структурираних података, као и извршавању прецизно дефинисаних упита. Иако *PostgreSQL* подржава претрагу пуног текста, таква претрага није његова примарна намјена и захтијева додатну конфигурацију и одговарајуће индексе. Код система чији је основни задатак претрага великих количина текстуалног садржаја, јавља се потреба за напреднијом анализом текста и ефикаснијим рангирањем резултата.

За разлику од релационих база, *Elasticsearch* је специјализован за претрагу и анализу текста. Он омогућава индексирање текстуалних докумената, њихову лексичку анализу и рангирање резултата према релевантности упита. Поред тога, подржава механизме који омогућавају флексибилнију претрагу, попут толерисања типографских грешака и различитих облика ријечи. Због тога се *Elasticsearch* показује као погодније рјешење за дио система који је задужен за претрагу великог корпуса текстуалних докумената, док релациона база и даље задржава своју улогу у чувању и управљању структурираним подацима.

6.3 Дистрибуирана архитектура

Elasticsearch је дистрибуиран систем који се састоји од једног или више чворова (енг. *nodes*) повезаних у кластер (енг. *cluster*). Чвор представља покренуту инстанцу *Elasticsearch-a*, док кластер представља групу чворова који заједнички управљају подацима и ресурсима система. Чворови који припадају истом кластеру идентификују се помоћу заједничког назива кластера (*cluster.name*). *Elasticsearch* може функционисати са само једним чвором, при чему такав чвор истовремено представља цијели кластер [22].

Фрагмент (енг. *Shard*) представља физичку јединицу у којој *Elasticsearch* складишти дио података једног индекса. Индекс је логичка цјелина која може бити састављена од једног или више фрагмената, док се сами

документи физички чувају и индексирају унутар фрагмената. Сваки *shard* представља засебну инстанцу претраживачког механизма и може се посматрати као самостална јединица за складиштење и претрагу [22].

Копија (енг. *Replica*) представља копију примарног фрагмента и служи као додатни ниво заштите података у случају отказа хардвера или недоступности чвора на којем се налази примарни фрагмент. Поред обезбјеђивања редундантности, копија може учествовати и у обради захтјева за читање, као што су претрага и преузимање докумената, чиме се може повећати капацитет система за операције читања [22].

Фрагмент и копија се постављају у конфигурацији индекса. У случају постављања *Elasticsearch-a* у дистрибуираном *Cloud* окружењу, уз коришћење *Kubernetes* кластера са више чворова, могуће је распоредити *Elasticsearch* чворове и фрагменте на више физичких или виртуелних машина. На тај начин различити фрагменти могу бити смјештени на различитим чворовима, па се упит који обухвата више фрагмената може обрађивати паралелно. Оваква архитектура омогућава хоризонтално скалирање система и расподјелу оптерећења на више рачунарских ресурса, што може побољшати перформансе претраге код великих количина података и већег броја истовремених захтјева. У оквиру овог мастер рада *Elasticsearch* се извршава локално на једном чвору, што је довољно за развој и тестирање система.

6.4 Како *Elasticsearch* чува и претражује податке?

Један од основних концепата на којем се заснива ефикасна претрага текста у *Elasticsearch-у* јесте инвертовани индекс (енг. *inverted index*) [22]. За разлику од класичног приступа, у којем се полази од документа и у њему траже одговарајуће ријечи, инвертовани индекс успоставља обрнуту везу, односно за сваки термин чува информацију о документима у којима се тај термин појављује [23]. Ова структура се може упоредити са индексом на крају књиге, гдје се уз одређену ријеч наводе странице на којима се она појављује. На сличан начин, *Elasticsearch* за сваки термин памти скуп докумената који га садрже.

Када корисник пошаље упит, *Elasticsearch* не мора пролазити кроз сваки документ појединачно. Умјесто тога, претрага се врши над већ изграђеним индексом, што значајно смањује количину података коју је потребно обрадити. Поред информације о документима у којима се термин појављује, индекс може садржати и додатне информације, попут учесталости термина и његове позиције у документу. Ови подаци се користе за напреднију претрагу и одређивање релевантности резултата [23].

6.5 *BM25* и рангирање резултата

Проналажење докумената који садрже термине из корисничког упита представља само први корак лексичке претраге. Уколико више докумената садржи исте термине, потребно је одредити њихову релевантност и поређати их тако да се најрелевантнији резултати прикажу на почетку. У *Elasticsearch-у* се за ту намјену користи алгоритам *Okapi BM25* (енг. *Best Matching 25*), који представља подразумевани алгоритам за израчунавање релевантности докумената [24].

Приликом израчунавања скорa узимају се у обзир три основна фактора: учесталост термина у документу, ријеткост термина у цијелом корпусу и дужина документа. Термин који се чешће појављује у документу има већу тежину, али његов допринос није линеаран, чиме се спречава да велики број понављања исте ријечи непропорционално повећа релевантност [24].

BM25 узима у обзир и дужину документа у односу на просјечну дужину докумената у колекцији. Ова нормализација је значајна за транскрипте, јер дужи сегменти природно могу садржати већи број термина и тиме имати већу вјероватноћу подударана са упитом. Узимањем у обзир дужине документа смањује се предност коју би дужи документи иначе имали само због своје величине.

У имплементираном систему *BM25* представља основу за рангирање резултата лексичке претраге. Основни текстуални упит реализован је помоћу *match* упита над пољем *text*, а добијени документи се рангирају према скору који *Elasticsearch* израчунава на основу *BM25* алгоритма. При томе, додатни услови постављени у *must* дијелу упита учествују у израчунавању релевантности, док услови постављени у *filter* дијелу служе за ограничење скупа докумената и не утичу на *BM25* скор.

У овом раду *BM25* није додатно имплементиран као посебан алгоритам, већ се користи као дио механизма за рангирање који обезбеђује Elasticsearch. На тај начин лексичка претрага комбинује ефикасност инвертованог индекса са статистичким моделом релевантности, при чему се кориснику најприје приказују сегменти који најбоље одговарају постављеном упиту.

6.6 Ниво вршења индексирања

Једна од првих одлука приликом имплементације лексичке претраге односила се на избор нивоа индексирања података. Разматране су биле три могућности:

- **Један документ по транскрипцији видеа** - организација индекса у којем један документ представља цијели видео
- **Један документ по сегменту** - организација индекса у којем би транскрипцијски сегмент представљао засебан документ
- **Један документ по транскрипцији видеа са угњежденим сегментима** - организација индекса у којем би се сегменти чували унутар документа који представља цијели видео

Иако би индексирање на нивоу видеа било једноставније и омогућило природно рангирање резултата по видеу, такав приступ не би омогућио директно враћање прецизног временског тренутка на којем се тражени садржај појављује. Са друге стране, чување сегмената као засебних докумената омогућава да сваки резултат претраге директно садржи текст и одговарајућу временску ознаку, чиме се кориснику може приказати конкретан дио разговора и омогућити прелазак на одговарајући тренутак у видеу. Због тога је у овом раду изабрана **грануларност на нивоу транскрипцијског сегмента**, односно сваки сегмент представља један документ у Elasticsearch индексу. Ова одлука је значајна и за каснију имплементацију семантичке претраге, јер се векторска претрага такође заснива на мањим текстуалним јединицама изведеним из транскрипцијских сегмената. На тај начин оба приступа претрази могу радити над приближно истим нивоом текстуалне грануларности, што поједностављује њихово касније поређење и комбиновање. Трећа могућност - један документ по транскрипцији видеа са угњежденим сегментима - био је избачен гледајући са становишта перформанси. Овакав приступ је сложенији и спорији над великим документима.

6.7 Мапирање

У *Elasticsearch*-у се структура података дефинише помоћу мапирања (енг. *mapping*), који одређује тип и начин индексирања појединачних поља документа. Мапирање се може упоредити са шемом у релационим базама података, гдје структура табеле дефинише типове и карактеристике њених колона. На основу мапирања *Elasticsearch* одређује како ће се вриједности појединих поља чувати, анализирати и индексирати, што директно утиче на начин извршавања и претраге над тим пољима [22].

Приликом дефинисања мапирања *Elasticsearch* индекса потребно је одредити која поља треба индексирати и који тип података им одговара. Основна одлука је била да се текст транскрипцијских сегмената дефинише као *text* поље, јер представља примарни садржај над којим се врши пуна-текстуална претрага (енг. *full-text search*). Насупрот томе, идентификатори и поља која се користе за филтрирање, сортирање или груписање дефинисани су одговарајућим нетекстуалним типовима. На овај начин мапирање је прилагођено стварним потребама претраге, умјесто да се у *Elasticsearch* копира цјелокупна структура података која већ постоји у релационој бази.

Приликом избора поља узета је у обзир и улога релационе базе као примарног извора података. Поља која нису неопходна за претрагу, као што су опис епизоде, трајање, број прегледа, број свиђања, путања до датотеке и интернет адреса (енг. *Uniform Resource Locator, URL*), нису дуплирана у *Elasticsearch*-у. Дескрипција видеа није укључена у поља предвиђена за претрагу, јер је ријеч о метаподатку који не представља транскрибовани садржај епизоде, а његово укључивање, ради дужине, могло би утицати на релевантност резултата претраге. Посебно је значајно што се вриједности попут броја прегледа и броја свиђања могу мијењати током времена, па њихово задржавање у *PostgreSQL*-у избјегава непотребно

синхронизовање између два система. Након претраге, *Elasticsearch* враћа идентификаторе релевантних видеа, на основу којих се остали подаци могу дохватити из релационе базе ради приказа на корисничком интерфејсу.

Са друге стране, у *Elasticsearch* су укључена поља која су директно повезана са претрагом и приказом резултата: идентификатор видеа (*video_id*), идентификатор сегмента (*segment_id*), садржај сегмента (*text*), наслов видеа (*title*), временски почетак сегмента (*start*), временски крај сегмента (*end*), наслов канала (енг. *channel_title*) и вријеме објаве видеа (*published_at*). Поље *text*, представник садржаја сегмента, дефинисано је кроз тип податка „*text*“ и представља основно поље за претрагу транскрипта, док је наслов видеа (*title*), такође дефинисан као тип податка „*text*“ како би корисник могао претраживати и наслове епизода. Идентификатор видеа (*video_id*) је дефинисан као тип податка „*keyword*“, како би се користи за идентификацију и повезивање резултата са подацима у релационој бази података. Идентификатор сегмента (*segment_id*) служи за идентификацију и очување редослиједа сегмената, док временски почетак сегмента (*start*) и временски крај сегмента (*end*) омогућавају враћање тачног временског интервала унутар видеа. Поља наслов канала (енг. *channel_title*) и вријеме објаве видеа (*published_at*) су укључена како би се омогућило филтрирање и сортирање резултата по каналу и датуму објављивања.

На овај начин *Elasticsearch* индекс садржи само податке релевантне за претрагу, филтрирање и идентификацију резултата, док *PostgreSQL* остаје централно мјесто за трајно чување комплетних података о епизодама. Оваква подјела одговорности смањује непотребно дуплирање података и омогућава да се промјенљиви подаци ажурирају у релационој бази без потребе за њиховим поновним индексирањем у *Elasticsearch*-у.

Мапирање се поставља у конфигурацији индекса. На тај начин *Elasticsearch* добија информацију о структури докумената и начину на који треба да обрађује њихова поља приликом претраге. Дио конфигурације којим је дефинисано мапирање коришћено у овој имплементацији приказан је у листингу 7.


```

"mappings": {
  "properties": {
    "video_id": {
      "type": "keyword"
    },
    "segment_id": {
      "type": "integer"
    },
    "text": {
      "type": "text",
      "analyzer": "serbian_analyzer",
      "search_analyzer": "serbian_analyzer"
    },
    "title": {
      "type": "text",
      "analyzer": "serbian_analyzer",
      "search_analyzer": "serbian_analyzer"
    },
    "start": {
      "type": "float"
    },
    "end": {
      "type": "float"
    },
    "channel_title": {
      "type": "keyword"
    },
    "published_at": {
      "type": "date"
    }
  }
}

```

Листинг 7: Конфигурација мапирања *Elasticsearch* индекса

6.8 Изазови српског језика у *Elasticsearch*-у

Примјена *Elasticsearch*-а над корпусом српског језика доноси неколико језички-специфичних изазова, детаљно обрађених у поглављу 2:

- богату флексију (падеже и глаголска времена — нпр. облици „Косово“, „Косову“, „Косова“ и „Косовом“). Са морфолошке стране гледано, ове ријечи представљају исти појам, али подразумемијевано подешавање их третира као различите термине.
- присуство два писма (ћирилица и латиница)
- дијакритике
- непостојање уграђене подршке за стематизацију и лематизацију српског језика у оквиру *Elasticsearch*-а.

Ниједан од ових изазова није ријешен подразумемијеваним (енг. *default*) подешавањем система, због чега је за потребе овог рада развијен прилагођени анализатор (енг. *analyzer*), описан у наставку овог поглавља.

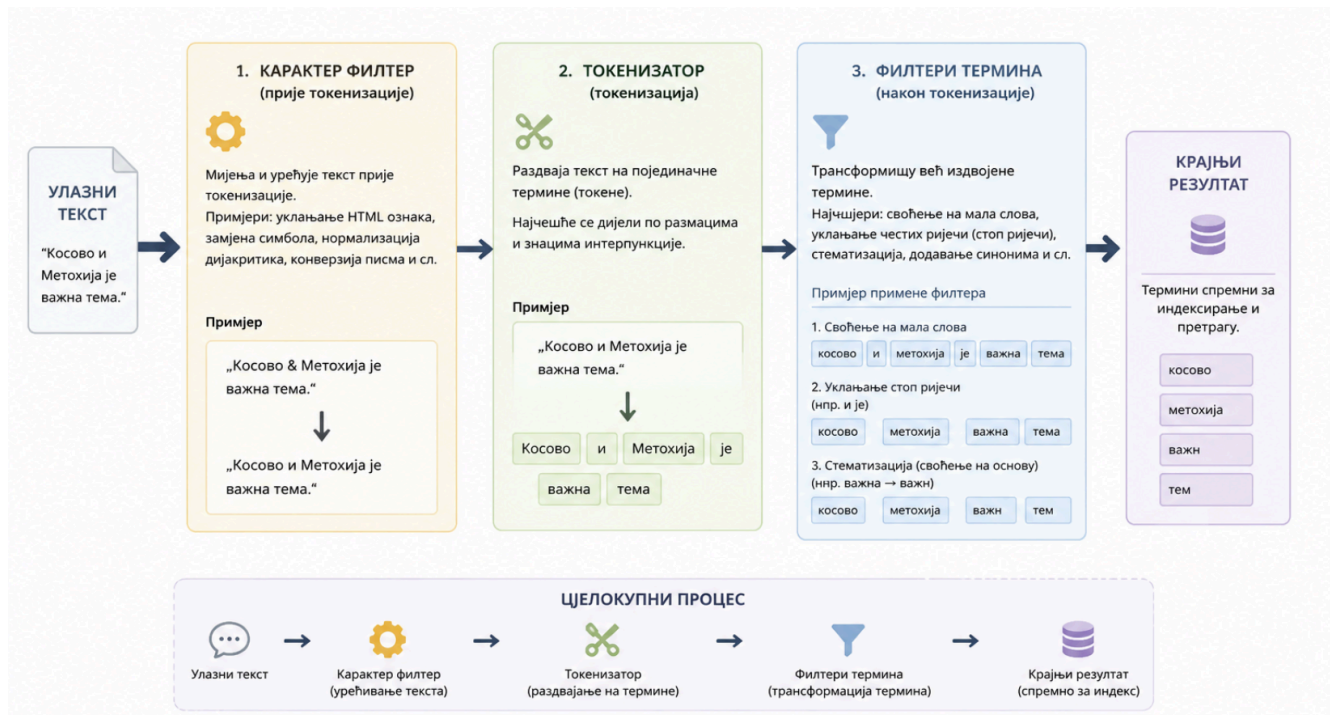
6.9 Анализатор

Анализатор (енг. *analyzer*) представља ланац трансформација кроз који пролази текст прије него што буде записан у обрнути индекс (при индексирању) или упоређен са садржајем индекса (при претраживању) [22]. Сваки анализатор се састоји од три компоненте:

- карактер филтера (енг. *character filter*) - мијења текст прије токенизације
- токенизатора (енг. *tokenizer*) - раздваја текст на појединачне термине

- низа филтера термина (енг. *token filter*) - трансформише већ издвојене термине, нпр. свођење на мала слова, уклањање честих ријечи, стематизацију и сл. [22].

На слици 3 приказан је поједностављен ток анализе текста кроз наведене компоненте анализатора.



Слика 3: Поједностављен приказ процеса анализе текста у Elasticsearch-у

Elasticsearch подразумевано нуди стандардни (енг. *standard*) анализатор, који представља опште рјешење за анализу текста на различитим језицима. Он раздваја текст на термине према границама ријечи, уклања већину знакова интерпункције и претвара термине у мала слова. Међутим, овакав приступ не узима у обзир специфичности појединачних језика, као што је њихова морфологија, зауставне ријечи (енг. *stop words*), те различите облике исте ријечи. Због тога стандардни анализатор није био довољан за потребе претраге корпуса на српском језику, већ је дефинисан прилагођени ланац филтера који омогућава адекватнију обраду српског текста.

6.9.1 Прилагођени анализатор за српски језик

За потребе лексичке претраге коришћен је прилагођени анализатор за српски језик. *Elasticsearch* поред стандардног анализатора пружа и језички специфичан српски (енг. *serbian*) анализатор, који уз стандардну токенизацију подржава уклањање зауставних ријечи (енг. *stop words*) и стематизацију карактеристичну за српски језик. Његова конфигурација може се реализовати и као прилагођени анализатор, што омогућава додавање додатних корака у процес анализе [25].

У првом кораку примјењује се **icu_folding филтер** из *ICU (International Components for Unicode)* додатка, који обезбјеђује *Unikod* (енг. *Universal Character Encoding, Unicode*) нормализацију и пресавијање карактера [26], што је посебно значајно за рад са дијакритичким знаковима. *ICU* додаток се користи за рјешавање проблема два писма ћирилице и латинице, тако да упит “Србија” и упит “Srbija” погађају исти термин у индексу. Након тога се примјењује **филтер lowercase**, којим се сви термини свде на мала слова, чиме се избјегава разликовање истих ријечи на основу величине слова.

Сљедећи корак представља уклањање зауставних ријечи (енг. *stop words*). *Elasticsearch* обезбјеђује уграђену листу српских зауставних ријечи означену са **serbian_stop**, коју је могуће користити у прилагођеном анализатору [27]. У оквиру имплементације коришћена је ова листа, а њена примјена је посебно значајна

за транскрипте, у којима се често појављују кратке функционалне ријечи које не доприносе значајно релевантности резултата. Примјер зауставних српских ријечи можете пронаћи у листингу 8 [28].

```
"ili", "a", "ali", "pa", "biti", "ne", "jesam", "sam", "jesi", "si",
"je", "jesmo", "smo", "jeste", "ste", "jesu", "su", "nijasam", "nisam", "nijas",
"nisi", "nije", "nijasmo", "nismo", "nije", "niste", "nijas", "nisu", "budem", "budeš",
"bude", "budemo", "budete", "budu", "budes", "bih", "bi", "bismo", "biste", "biše",
"bise", "bio", "bili", "budimo", "budite", "bila", "bilo", "bile", "ću", "ćeš",
"će", "ćemo", "ćete", "neću", "nećeš", "neće", "nećemo", "nećete", "cu", "ces",
"ce", "cemo", "cete", "necu", "neces", "nece", "necemo", "necete", "mogu",
"možeš", "može", "možemo", "možete", "mozes", "moze", "možemo", "mozete", "и", "или",
"а", "али", "па", "бити", "не", "јесам", "сам", "јеси", "си", "је",
"јесмо", "смо", "јесте", "сте", "јесу", "су", "нијесам", "нисам", "нијеси",
"ниси", "није", "нијесмо", "нисмо", "нијесте", "нисте", "нијесу", "нису", "будем", "будеш",
"буде", "будемо", "будете", "буду", "будес", "бих", "би", "бисмо", "бисте",
"бише", "бисе", "био", "били", "будимо", "будите", "била", "било", "биле", "ћу",
"ћеш", "ће", "ћемо", "ћете", "нећу", "нећеш", "неће", "нећемо", "нећете", "цу",
"цес", "це", "цемо", "цете", "нецу", "нецес", "неце", "нецемо", "нецете", "могу",
"можеш", "може", "можемо", "можете", "мозес", "мозе", "можемо", "можете"
```

Листинг 8: Списак зауставних ријечи у уграђеној листи *serbian_stop*

За рјешавање морфолошке варијативности српског језика примјењује се ***serbian_stemming***. *Elasticsearch* за српски језик обезбјеђује стемер који се може користити као ***stemmer филтер*** са параметром ***language: "serbian"*** [29]. На овај начин различити облици исте ријечи могу бити сведени на заједнички облик, чиме се повећава вјероватноћа да ће упит написан у једном граматичком облику пронаћи документе у којима се појављује други облик исте ријечи. На примјер, претрага по термину „Косово“ може обухватити и појављивања облика „Косову“ или „Косова“. Коначан ланац анализе коришћен за поља *text* и *title* приказан је на листингу 9.

```
"analysis": {
  "analyzer": {
    "serbian_analyzer": {
      "type": "custom",
      "tokenizer": "standard",
      "filter": [
        "icu_folding",
        "lowercase",
        "serbian_stop",
        "serbian_stemming"
      ]
    }
  },
  "filter": {
    "serbian_stop": {
      "type": "stop",
      "stopwords": "_serbian_"
    },
    "serbian_stemming": {
      "type": "stemmer",
      "language": "serbian"
    }
  }
}
```

Листинг 9: Конфигурација прилагођеног анализатора за српски језик

6.9.2 Скраћеница: два анализатора, за индексирање и за претрагу

Ланац филтера описан у претходном одјелу не рјешава један специфичан проблем — скраћенице и акрониме. Корисник који упише „КиМ“ очекује резултате који садрже пуни израз „Косово и Метохија“, иако та два низа знакова немају ниједан заједнички термин: ниједна стематизација ни нормализација писма то не премошћува. Проблем се не рјешава ни семантичком (векторском) претрагом (Глава 7). Скраћенице представљају слијепу тачку (енг. *blind spot*) *embedding* модела, јер модел вјероватно није, или је ријетко, видео дати акроним током тренирања, те за њега не постоји поуздана векторска репрезентација, за разлику од појмовних синонима које векторски модел углавном препознаје и без експлицитне листе.

Рјешење је филтер за синониме (енг. *synonym token filter*), који при обради текста додаје алтернативне облике термина. Кључна одлука тиче се тренутка у ком се синоними примјењују: при индексирању (енг. *index time*) или при претраживању (енг. *search time*). Да су синоними примијењени при индексирању, свака измјена листе скраћеница захтијевала би поновно индексирање цијелог корпуса — у случају овог система, реиндексирање преко девет и по милиона докумената, у трајању преко 60 минута. С обзиром на то да се листа скраћеница очекивано мијења током даљег рада и тестирања система, одабрана је примјена искључиво при претраживању.

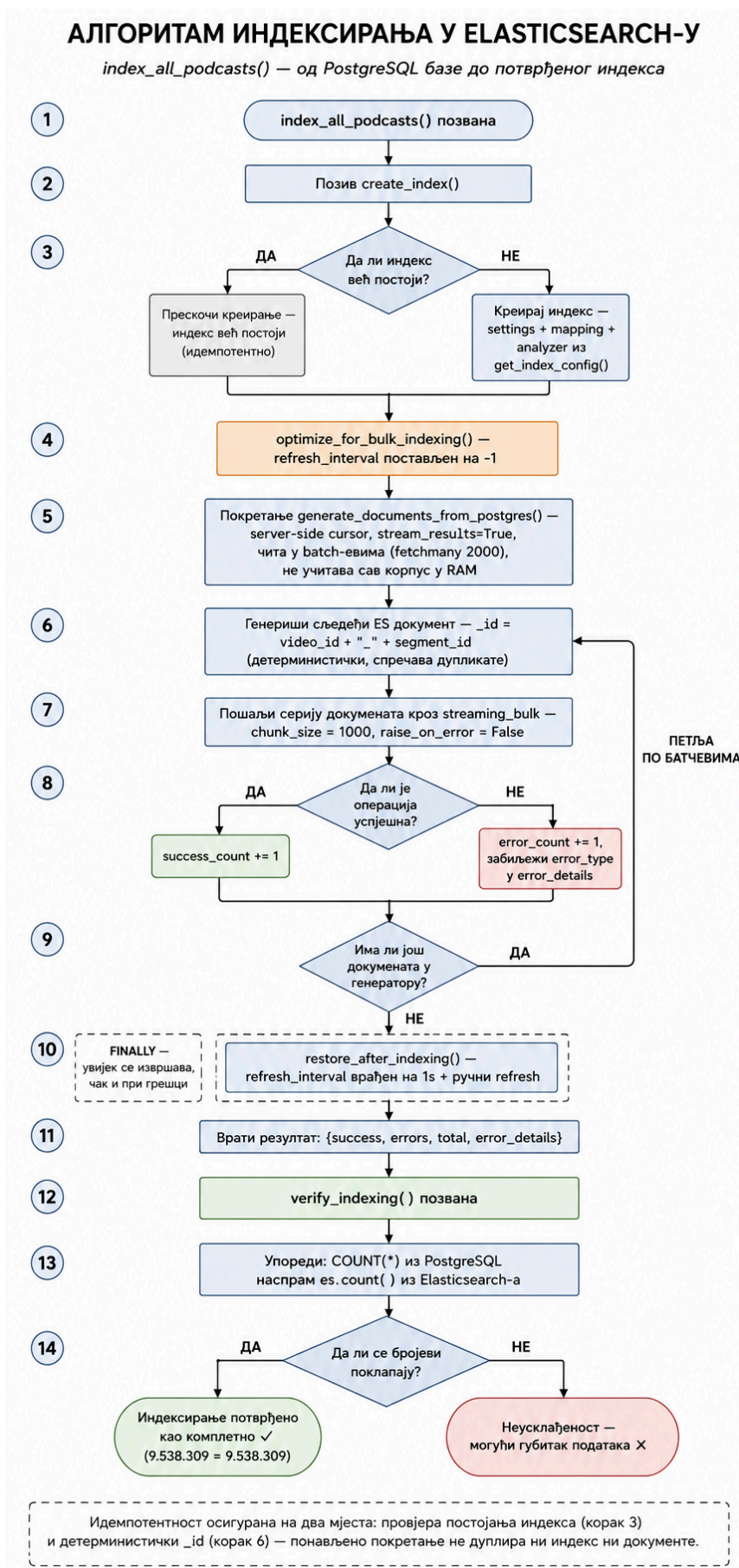
Ова одлука захтијева дефинисање два одвојена анализатора: индексни анализатор (нпр. `serbian_analyzer`), кориштен приликом индексирања и наведен директно у мапирању поља `text` и `title`, и претраживачки анализатор (`serbian_search_analyzer`), који садржи исти ланац филтера уз додатак филтера за синониме, а који се експлицитно наводи као параметар `analyzer` у самом упиту за претрагу, умјесто да буде дио мапирања. Захваљујући овом раздвајању, додавање нове скраћенице своди се на измјену подешавања индекса (затварање индекса, ажурирање подешавања, отварање индекса) — операцију која траје неколико секунди, без потребе за реиндексирањем докумената. Листа имплементираних скраћеница релевантних за корпус дата је у листингу 10.

```
SYNONYMS = [
    "kim, kosovo i metohija, kosmet",
    "bih, bosna i hercegovina",
    "rs, republika srpska",
    "cg, crna gora, montenegro",
    "sad, sjedinjene americke drzave, amerika, usa",
    "eu, evropska unija",
    "nato, sjeverno-atlantski savez",
    "un, ujedinjene nacije",
    "mmf, medjunarodni monetarni fond",
    "rts, radio televizija srbije",
    "sns, srpska napredna stranka",
    "sps, socijalisticka partija srbije",
    "bdp, bruto domaci proizvod",
]
```

Листинг 10: Списак скраћеница коришћених у *Elasticsearch* претрази

6.10 Алгоритам индексирања

Цјелокупан поступак индексирања — од провјере постојања индекса, преко генерисања и слања документа, до финалне верификације — приказан је као дијаграм тока на слици 4, а детаљно је образложен у наставку овог одјелка.



Слика 4: Алгоритам индексирања у *Elasticsearch*-у — од провјере постојања индекса до верификације учитаних докумената

6.10.1 Креирање индекса

Функција `create_index()` прије креирања провјерава да ли индекс са датим именом већ постоји (корак 3 на слици 4) — та провјера чини операцију идемпотентном: позивање функције више пута (нпр. приликом поновног покретања *backend* сервиса) неће обрисати нити дуплирати постојећи индекс. Уколико индекс не постоји, креира се позивом Elasticsearch API-ја са комплетном конфигурацијом добијеном из `get_index_config()` — подешавањима, мапирањем и *analyzer* ланцем описаним у претходним одјељцима.

6.10.2 Bulk индексирање

Индексирање преко девет и по милиона докумената захтијева другачији приступ од упућивања појединачног захтјева за сваки документ. У наставку је образложено зашто је коришћен *Bulk API* умјесто индексирања документ-по-документ. Bulk API (енг. *bulk*) представља *Elasticsearch API endpoint* који омогућава извршавање више операција индексирања, креирања, брисања или ажурирања докумената у оквиру једног *HTTP* захтјева, чиме се смањује мрежни трошак и значајно повећава брзина индексирања [30]. Потом сљедују објашњења конкретне оптимизација и тока извршавања самог поступка (кораки 4 - 11 на слици 4).

6.10.2.1 Зашто *bulk*, а не документ-по-документ

Слање једног *HTTP* захтјева за сваки појединачни документ носи фиксни трошак (успостављање везе, парсирање захтјева, координација унутар кластера) који се, при обиму од преко девет и по милиона докумената, вишеструко умножава и чини индексирање непрактично спорим. *Elasticsearch* за то нуди *Bulk API* [30] — један *HTTP* захтјев који садржи низ операција индексирања, чиме се фиксни трошак по захтјеву амортизује на цијелу серију докумената умјесто на сваки појединачно.

Да би се читав корпус проследио *Bulk API*-ју без исцрпљивања *RAM* меморије, документи се не учитавају одједном у листу, него генеришу један по један помоћу функције `generate_documents_from_postgres()` (корак 5 на слици 4).

Идентификатор сваког документа се експлицитно поставља на `f"{video_id}_{segment_id}"` (корак 6), умјесто да га *Elasticsearch* генерише аутоматски. Овим се индексирање чини идемпотентним: уколико се поступак индексирања понови (нпр. после прекида или измјене у подацима), документ са истим *video_id* и *segment_id* је замијенjen (*upsert*), а не дуплиран.

6.10.2.2 Оптимизација и покретање

Функција `optimize_for_bulk_indexing()` привремено искључује освјежавање индекса (*refresh_interval*), подешавајући га на `-1` за вријеме трајања *bulk* индексирања (корак 4 на слици 4). Подразумијевано, *Elasticsearch* освјежава индекс на сваку секунду како би нови документи одмах били доступни за претрагу — при индексирању милиона докумената у кратком временском периоду то освјежавање непотребно троши ресурсе, јер документи не морају бити тренутно претраживи док траје почетно пуњење индекса. По завршетку индексирања, *refresh_interval* се враћа на уобичајену вриједност (корак 10, извршен у *finally* блоку — гарантовано се извршава чак и уколико дође до грешке током индексирања). Ова оптимизација одговара званичној Elastic препоруци за убрзавање индексирања при почетном пуњењу великих индекса [31].

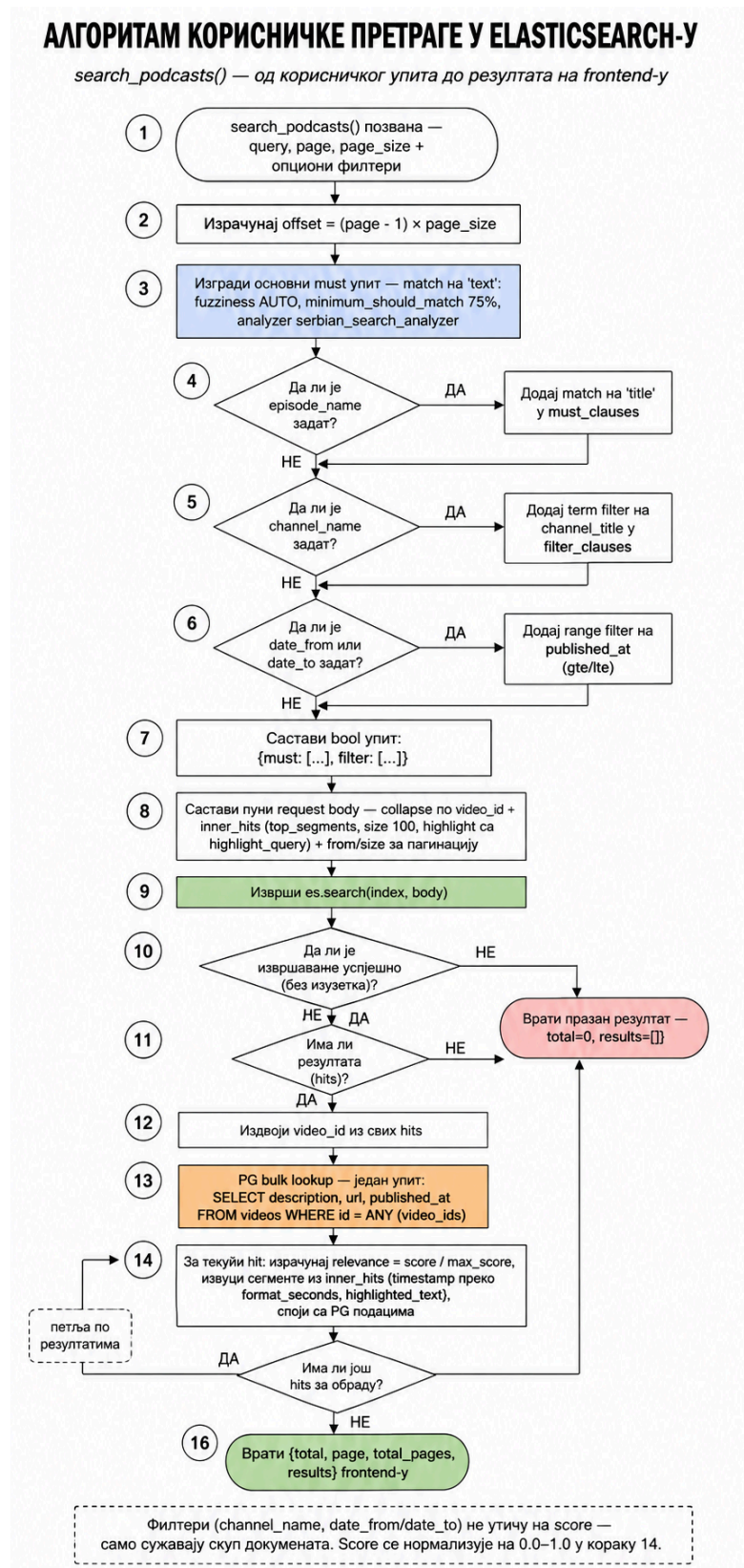
Поступак је покретан преко административног *endpoint*-а (*POST /admin/index/create*), након што су провјерени */health* статуси базе и *Elasticsearch* кластера. Резултат покретања над цјелокупним корпусом дат је у листингу 11.

```
{"success": 9538309, "errors": 0, "total": 9538309}
```

Листинг 11: Резултат *bulk* индексирања докумената

6.11 Алгоритам претраге

Док претходни одјелјак описује једнократни поступак попуњавања индекса, овдје је приказан ток који се извршава при свакој корисничкој претрази — од примљеног упита, преко грађења *Elasticsearch* упита, до формирања одговора за корисничког интерфејса. Комплетан ток приказан је на слици 5.



Слика 5: Алгоритам корисничке претраге у *Elasticsearch*-у

6.11.1 Грађење *Elasticsearch* упита

Функција `search_podcasts()` прима текстуални упит (енг. *query*) као обавезан параметар, те четири опциона филтера — назив епизоде, назив канала и опсег датума — уз параметре страничења *page* и *page_size* (корак 1 на слици 5).

Основни дио упита је увијек исти: **match** над пољем *text*, при чему **fuzziness**: “**AUTO**” омогућава толерирање мањих правописних грешака (*minimum_should_match*: “75%”), а *analyzer*: “*serbian_search_analyzer*” обезбјеђује анализу прилагођену српском језику. Упит *match* представља стандардни начин за пуну текстуалну претрагу у *Elasticsearch*-у [32]. Основни дио упита приказан је у листингу 12.

```
must_clauses = [
    {
        "match": {
            "text": {
                "query":          query,
                "fuzziness":      "AUTO",
                "minimum_should_match": "75%",
                "analyzer":      "serbian_search_analyzer"
            }
        }
    }
]
```

Листинг 12: Конфигурација *match* упита за претрагу текста у *Elasticsearch*-у

Три опциона услова додају се у упит само уколико их је корисник заиста задао (кораци 4–6). Назив епизоде се додаје као додатни **match** у *must_clauses*, док назив канала и опсег датума иду у *filter_clauses*. Параметри у *must* дијелу упита утичу на *BM25* скор релевантности, док параметри у *filter* дијелу само искључују документе из резултата, без утицаја на рангирање.

6.11.2 Collapse, inner_hits и highlight

Прије слања упита, *query* се уграђује у комплетан *request body* (корак 8). Два додатна механизма овдје су кључна за понашање претраге:

- **collapse** по пољу *video_id* осигурава да се сваки видео у резултатима прикаже само једном. За сваки видео приказује се један главни резултат, док се до 100 најрелевантнијих сегмената тог видеа издваја посебно, у оквиру *inner_hits*.
- **highlight** конфигурација означава подударачуће дијелове текста *mark* ознакама.

6.11.3 Извршавање, *PG bulk lookup* и формирање одговора

Позив `es.search()` (корак 9 на слици 5) обавијен је *try/except* блоком — уколико упит из некаквог разлога не успије, враћа се празан резултат (*total: 0, results: []*) умјесто да цијела претрага откаже (корак 10). Исти празан резултат враћа се и уколико упит успије, али не пронађе ниједан одговарајући документ (корак 11).

Уколико резултата има, *video_id* вриједности свих враћених документа издвајају се у јединствену листу (корак 12), над којом се извршава један упит ка *PostgreSQL* бази — исти принцип *bulk lookup*-а који је већ примјењен приликом индексирања. Овдје га примјењујемо у супротном смјеру: један упит за опис, интернет адресу и датум објављивања свих видеа из резултата, умјесто једног упита по видеу (кораци 12–13), приказ исјечка кода кроз листинг 13.


```

video_ids = [hit["_source"]["video_id"] for hit in hits]

result = conn.execute(
    text("""
        SELECT id, description, url, published_at
        FROM videos
        WHERE id = ANY(:ids)
    """),
    {"ids": video_ids}
)
pg_data = {row.id: row for row in result}

```

Листинг 13: Дохватање метаподатака релационе базе

За сваки резултат затим се рачуна нормализован скор релевантности, дијелењем сировог (енг. raw) *ES* скорa максималним скором у скупу резултата (***round(hit_score / max_score, 2)***), а временске ознаке сегмената претварају се из секунди у читљив формат (***format_seconds()***, нпр. ***83.4*** → ***“1:23”***) — ова петља понавља се за сваки резултат (кораци 14–15), приказан исјечак кода у листингу 14.

```

relevance = round((hit["_score"] / max_score), 2)

matches = []
for seg_hit in hit["inner_hits"]["top_segments"]["hits"]["hits"]:
    seg = seg_hit["_source"]
    highlighted = seg_hit.get("highlight", {}).get(
        "text", [seg["text"]]
    )[0]

    matches.append({
        "timestamp": format_seconds(seg["start"]),
        "timestamp_seconds": int(seg["start"]),
        "text": seg["text"],
        "highlighted_text": highlighted
    })

```

Листинг 14: Нормализација скорa релевантности и форматирање временских ознака сегмената

На крају, *ES* подаци (наслов, канал, сегменти, релевантност) спајају се са *PostgreSQL* подацима (опис, интернет адреса, датум објаве) у јединствен одговор који се просљеђује функцији хибридизације (поглавље 8).

Глава 7

Семантичка (векторска) претрага

Глава 8

Хибридизација резултата претраге (RRF)

У закључку дајте кратак преглед онога шта урађено, са освртом на проблеме који су решени, предности и мане решења и правце даљег развоја.

Списак слика

Слика 1	Дијаграм класа модела података: Channel, Video, TranscriptSegment и Chunk.	16
Слика 2	Алгоритам увоза података из <i>.jsonl.gz</i> датотека у PostgreSQL базу.	17
Слика 3	Поједностављен приказ процеса анализе текста у Elasticsearch-у	24
Слика 4	Алгоритам индексирања у <i>Elasticsearch</i> -у — од провјере постојања индекса до верификације учитаних докумената	27
Слика 5	Алгоритам корисничке претраге у <i>Elasticsearch</i> -у	29

Списак листинга

Листинг 1	Конфигурација <i>PostgreSQL</i> базе са <i>pgvector</i> екстензијом	9
Листинг 2	<i>Dockerfile</i> за изградњу прилагођене <i>Elasticsearch</i> слике са <i>analysis-icu</i> додатком	10
Листинг 3	Конфигурација <i>nginx</i> -а за просљеђивање захтјева <i>Angular SPA</i> апликацији	10
Листинг 4	Команде за покретање и заустављање <i>Docker Compose</i> сервиса	11
Листинг 5	Примјер структуре JSON записа са метаподацима и транскрипцијом епизоде	14
Листинг 6	Покретање скрипте за увоз података унутар <i>backend</i> контејнера	16
Листинг 7	Конфигурација мапирања <i>Elasticsearch</i> индекса	23
Листинг 8	Списак зауставних ријечи у уграђеној листи <i>serbian_stop</i>	25
Листинг 9	Конфигурација прилагођеног анализатора за српски језик	25
Листинг 10	Списак скраћеница коришћених у <i>Elasticsearch</i> претрази	26
Листинг 11	Резултат <i>bulk</i> индексирања докумената	28
Листинг 12	Конфигурација <i>match</i> упита за претрагу текста у <i>Elasticsearch</i> -у	30
Листинг 13	Дохватање метаподатака релационе базе	31
Листинг 14	Нормализација скорa релевантности и форматирање временских ознака сегмената	31

Списак табела

Табела 1	Списак канала у корпусу и број транскрибованих епизода по каналу.	13
----------	--	----

Списак коришћених скраћеница

Скраћеница	Опис
ANN	Approximate Nearest Neighbor (приближна претрага к-најближих суседа)
NLP	Natural Language Processing (обрада природног језика)
RRF	Reciprocal Rank Fusion (метода спајања рангираних листи резултата)

Списак коришћених појмова

Појам	Објашњење
Подкаст	Аудио (или аудио-видео) садржај који се дистрибуира у виду епизода, доступан за преузимање или стриминг путем интернета.
Семантичка претрага	Начин претраге који упоређује значење упита и садржаја, а не само подударање тачних термина.
Лексичка претрага	Начин претраге заснован на подударању тачних ријечи или термина између упита и садржаја (нпр. кроз индексирање кључних речи).
Хибридна претрага	Пристап претраживању који комбинује лексичку и семантичку претрагу ради постизања прецизнијих резултата.
Гранулација сегмената	Ниво детаљности на који је садржај подјелењен (нпр. на један или више сегмената) ради навигације до тачног или приближног тренутка унутар епизоде.
Унакрсни језички пренос знања	Пренос научених представа или образаца из модела тренираног на једном (обично доминантном) језику на други, мање заступљен језик.
Референтни тест (benchmark)	Стандардизован скуп задатака и података који омогућава упоредиво мјерење перформанси различитих система или модела.

Биографија

Татјана Гавриловић је рођена 22.07.1999. године у Милићима, Република Српска, Босна и Херцеговина. Дјетињство је провела у Дервенти, гдје је 2014. године завршила основну школу “19. Април” као носилац Вукове дипломе. Исте године уписала је Гимназију општег смјера у Дервенти, коју је завршила 2018. године, поново као носилац Вукове дипломе и ученик генерације. Током школовања учествовала је на бројним општинским и регионалним такмичењима из математике, физике и историје.

У јулу 2018. године уписала је Факултет техничких наука у Новом Саду, смјер Софтверско инжењерство и информационе технологије, који је завршила 2022. године одбраном рада на тему “Имплементација Analyze me - Write me апликације употребом *Mendix* развојног окружења”.

Радно искуство стицала је кроз двије стручне праксе — у компанији Intermino у Новом Саду (јул – август 2022, Salesforce) и у компанији Vega IT (новембар 2022), на позицији софтверског инжењера у области веб програмирања. Од 2023. године запослена је у компанији Openfyle на позицији софтверског инжењера, гдје ради и данас. До 2025. године радила је и као сарадник у настави на Факултету техничких наука у Новом Саду, гдје је држала вјежбе из предмета Основе програмирања, Инжењерство серверског слоја и Пословне информатике — при чему је, слично као и на дипломском раду, у оквиру Пословне информатике коришћена *Mendix* развојна платформа.

Мастер студије, смјер Софтверско инжењерство и информационе технологије, уписала је у септембру 2023. године на Факултету техничких наука у Новом Саду. Говори енглески језик. Њена стручна интересовања обухватају системе за претрагу, модел-управљане (енг. *model-driven*) системе, системе засноване на знању и правилима (енг. *rule-based*), као и моделовање софтвера уопште. Интересовање за модел-управљене приступе дијелом произилази из искуства стеченог кроз рад са *Mendix* платформом, и на дипломском раду, у оквиру ког је развила апликацију намјењену писцима аматерима за подршку у писању књижевног романа, од почетне идеје до завршеног дјела.

Литература

- [1] A. Tamborrino, “Introducing Natural Language Search for Podcast Episodes.” Accessed: August 19, 2026. [Online]. Доступно на <https://engineering.atspotify.com/2022/03/introducing-natural-language-search-for-podcast-episodes>
- [2] J. Karlgren *et al.*, “TREC 2021 Podcasts Track Overview,” in *Proceedings of the Thirtieth Text REtrieval Conference (TREC 2021)*, National Institute of Standards, Technology (NIST), 2021. doi: [10.6028/NIST.SP.500-335.podcast-overview](https://doi.org/10.6028/NIST.SP.500-335.podcast-overview).
- [3] “TREC Podcasts Track.” Accessed: August 19, 2026. [Online]. Доступно на <https://trecpodcasts.github.io/>
- [4] “Семантичка претрага судске праксе.” Accessed: August 19, 2026. [Online]. Доступно на <https://sudskapraksa.rs/>
- [5] ICEF and R. Centar, “COMtext.SR.” Accessed: August 19, 2026. [Online]. Доступно на <https://icef-nlp.github.io/COMtext.SR/>
- [6] S. A. Ubois, “From Data Scarcity to Data Care: Reimagining Language Technologies for Serbian and other Low-Resource Languages,” *arXiv preprint arXiv:2512.10630*, 2025, doi: [10.48550/arXiv.2512.10630](https://doi.org/10.48550/arXiv.2512.10630).
- [7] V. Bataonović, “Методологија рјешавања семантичких проблема у обради кратких текстова написаних на природним језицима са ограниченим ресурсима,” Doctoral dissertation, 2020. Accessed: August 23, 2026. [Online]. Доступно на <https://nardus.mpn.gov.rs/bitstream/handle/123456789/17783/Disertacija.pdf?sequence=1&isAllowed=y>
- [8] M. Benjamin, “Hard Numbers: Language Exclusion in Computational Linguistics and Natural Language Processing.” Accessed: August 23, 2026. [Online]. Доступно на http://lrec-conf.org/workshops/lrec2018/W26/pdf/23_W26.pdf
- [9] L. Tang and T. Vuković, “NLP for preserving Torlak, a vulnerable low-resource Slavic language,” in *Proceedings of the 31st International Conference on Computational Linguistics (COLING 2025)*, 2025. Accessed: August 23, 2026. [Online]. Доступно на <https://aclanthology.org/2025.coling-main.423.pdf>
- [10] Elastic, “ICU analysis plugin.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/docs/reference/elasticsearch/plugins/analysis-icu>
- [11] Одсек за стандардни језик, “Морфологија.” Accessed: August 23, 2026. [Online]. Доступно на <https://www.lingvistickitermini.rs/pojmovnik/morfologija/>
- [12] Одсек за стандардни језик, “Лексема.” Accessed: August 23, 2026. [Online]. Доступно на <https://www.lingvistickitermini.rs/pojmovnik/leksema/>
- [13] Docker, “Defining and running multi-container applications with Docker Compose.” Accessed: August 23, 2026. [Online]. Доступно на <https://docs.docker.com/compose/>
- [14] pgvector, “pgvector: Open-source vector similarity search for Postgres.” Accessed: August 23, 2026. [Online]. Доступно на <https://github.com/pgvector/pgvector>
- [15] Elastic, “Download X-Pack: Extend Elasticsearch and Kibana.” Accessed: August 23, 2026. [Online]. Доступно на <https://www.elastic.co/downloads/x-pack>
- [16] FastAPI, “FastAPI documentation.” Accessed: August 23, 2026. [Online]. Доступно на <https://fastapi.tiangolo.com/>

-
- [17] Angular, “What is Angular?.” Accessed: August 23, 2026. [Online]. Доступно на <https://angular.dev/overview>
- [18] Docker, “Multi-stage builds.” Accessed: August 23, 2026. [Online]. Доступно на <https://docs.docker.com/build/building/multi-stage/>
- [19] J. R. Batista, *Learn OpenAI Whisper: Transform your understanding of GenAI through robust and accurate speech processing solutions*. Packt Publishing, 2024, p. 372.
- [20] A. Jain, “TF-IDF in NLP (Term Frequency Inverse Document Frequency).” Accessed: August 21, 2026. [Online]. Доступно на <https://medium.com/@abhishekjainindore24/tf-idf-in-nlp-term-frequency-inverse-document-frequency-e05b65932f1d>
- [21] S. Connelly, “Practical BM25 — Part 2: The BM25 Algorithm and its Variables.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/blog/practical-bm25-part-2-the-bm25-algorithm-and-its-variables>
- [22] C. Gormley and Z. Tong, *Elasticsearch: The Definitive Guide: A Distributed Real-Time Search and Analytics Engine*. O'Reilly Media, 2015. [Online]. Доступно на https://hlszny.com/booksAndPapers/buckets/b8_IT/elasticsearch-the-definitive-guide.pdf
- [23] Elastic, “How Full-Text Search Works.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/docs/solutions/search/full-text/how-full-text-works>
- [24] Elastic, “Similarity module.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/guide/en/elasticsearch/reference/current/index-modules-similarity.html>
- [25] Elastic, “Language analyzers.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/docs/reference/text-analysis/analysis-lang-analyzer#serbian-analyzer>
- [26] Elastic, “ICU analyzer.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/docs/reference/elasticsearch/plugins/analysis-icu-analyzer>
- [27] Elastic, “Stop token filter.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/docs/reference/text-analysis/analysis-stop-tokenfilter>
- [28] Apache Lucene, “Serbian stop words (stopwords.txt).” Accessed: August 21, 2026. [Online]. Доступно на <https://github.com/apache/lucene/blob/main/lucene/analysis/common/src/resources/org/apache/lucene/analysis/sr/stopwords.txt>
- [29] Elastic, “Stemmer token filter.” Accessed: August 21, 2026. [Online]. Доступно на <https://www.elastic.co/docs/reference/text-analysis/analysis-stemmer-tokenfilter>
- [30] Elastic, “Bulk index or delete documents.” Accessed: August 22, 2026. [Online]. Доступно на <https://www.elastic.co/guide/en/elasticsearch/reference/current/docs-bulk.html>
- [31] Elastic, “Tune for indexing speed.” Accessed: August 22, 2026. [Online]. Доступно на <https://www.elastic.co/docs/deploy-manage/production-guidance/optimize-performance/indexing-speed>
- [32] Elastic, “Match query.” Accessed: August 22, 2026. [Online]. Доступно на <https://www.elastic.co/guide/en/elasticsearch/reference/8.19/query-dsl-match-query.html>